



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Rodrigo Alejandro Barbosa Vidal

Fecha:
12/06/26

Introducción

La arquitectura de software es la estructura que define cómo se organizan y relacionan los componentes de un sistema. Elegir una arquitectura adecuada es importante porque influye en el rendimiento, mantenimiento, escalabilidad y calidad del software.

1.-INVESTIGACIÓN

MVC (Model-View-Controller)

Definición

Es un patrón que divide una aplicación en tres componentes principales para separar responsabilidades y facilitar el mantenimiento.

Estructura general

Se compone de Modelo, Vista y Controlador.

Componentes principales

Modelo: Gestiona los datos.

Vista: Muestra la información al usuario.

Controlador: Recibe las acciones del usuario y coordina la comunicación entre Modelo y Vista.

Forma de funcionamiento

El usuario interactúa con la Vista, el Controlador procesa la solicitud y actualiza el Modelo.

Después, la vista muestra la información actualizada.

Ventajas

Organización clara del código, facilita el mantenimiento, permite reutilizar componentes, favorece el trabajo en equipo.

Desventajas

Puede resultar complejo para proyectos pequeños.

Requiere una correcta separación de responsabilidades.

Casos de uso

Aplicaciones web.

Sistemas administrativos.

Plataformas educativas.

Arquitectura en Capas (Layered Architecture)

Definición

Organiza el sistema en diferentes niveles o capas, donde cada una tiene una función específica.

Estructura general

Presentación, lógica de negocio y acceso a datos.

Componentes principales

Capa de presentación.

Capa de negocio.

Capa de datos.

Forma de funcionamiento

Cada capa se comunica únicamente con la capa inmediata inferior o superior.

Ventajas

Fácil mantenimiento.

Buena organización.

Permite reutilizar componentes.

Desventajas

Puede afectar el rendimiento al pasar por varias capas.

Cambios importantes pueden impactar varias capas.

Casos de uso

Sistemas empresariales.

Aplicaciones administrativas.

Sistemas escolares.

Microservicios

Definición

Es una arquitectura donde la aplicación se divide en pequeños servicios independientes que trabajan juntos.

Estructura general

Cada servicio tiene una función específica y puede ejecutarse de forma independiente.

Componentes principales

Servicios independientes.

API de comunicación.

Base de datos propia o compartida.

Forma de funcionamiento

Los servicios se comunican mediante APIs o servicios web.

Ventajas

Alta escalabilidad.
Fácil actualización de componentes.
Mayor tolerancia a fallos.

Desventajas

Mayor complejidad.
Requiere más recursos y experiencia.
Comunicación más difícil entre servicios.

Casos de uso

Netflix.
Amazon.
Plataformas de streaming.
Grandes sistemas empresariales.

Arquitectura Monolítica

Definición

Toda la aplicación se desarrolla y despliega como una sola unidad.

Estructura general

Todos los módulos están integrados en un único sistema.

Componentes principales

Interfaz de usuario.
Lógica de negocio.
Acceso a datos.

Forma de funcionamiento

Todos los componentes funcionan dentro de la misma aplicación.

Ventajas

Desarrollo sencillo.
Fácil implementación inicial.
Menor complejidad.

Desventajas

Difícil escalabilidad.

Mantenimiento complicado cuando el sistema crece.
Un fallo puede afectar a todo el sistema.

Casos de uso

Proyectos pequeños.
Aplicaciones con pocos usuarios.
Sistemas en etapas iniciales.

Cliente Servidor

Definición

Modelo donde un cliente solicita servicios y un servidor responde a dichas solicitudes.

Estructura general

Cliente conectado a uno o varios servidores.

Componentes principales

Cliente.
Servidor.
Red de comunicación.

Forma de funcionamiento

El cliente realiza una petición y el servidor procesa la solicitud y devuelve una respuesta.

Ventajas

Centralización de información.
Fácil administración.
Seguridad más controlada.

Desventajas

Dependencia del servidor.
Posibles cuellos de botella.
Si el servidor falla, afecta a todos los clientes.

Casos de uso

Sistemas bancarios.
Aplicaciones web.
Sistemas empresariales.

2.-CUADRO COMPARATIVO

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar responsabilidades	Organizar funciones por niveles	Unificar todo en una aplicación	Dividir el sistema en servicios independientes	Permitir comunicación entre cliente y servidor
Estructura	Modelo, Vista y Controlador	Capas jerárquicas	Aplicación única	Servicios independientes	Cliente y Servidor
Organización del código	Muy organizada	Organizada por capas	Centralizada	Distribuida	Centralizada en servidor
Escalabilidad	Media	Media	Baja	Alta	Media
Mantenimiento	Fácil	Fácil	Difícil cuando crece	Fácil por servicios	Moderado
Complejidad	Media	Baja	Baja	Alta	Media
Facilidad de implementación	Alta	Alta	Muy alta	Baja	Media
Rendimiento	Bueno	Bueno	Muy bueno	Bueno, depende de la comunicación	Bueno
Reutilización	Alta	Alta	Baja	Alta	Media
Casos de uso recomendados	Aplicaciones web	Sistemas empresariales	Proyectos pequeños	Grandes plataformas	Aplicaciones conectadas a red
Ventajas	Orden y mantenimiento	Organización clara	Simplicidad	Escalabilidad y flexibilidad	Centralización de recursos

Desventajas	Más estructura inicial	Puede generar más procesos	Difícil crecimiento	Complejo de administrar	Dependencia del servidor
-------------	------------------------	----------------------------	---------------------	-------------------------	--------------------------

3.- ANÁLISIS

1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

MVC es el patrón más adecuado porque permite organizar claramente el código y facilita el mantenimiento. Además, es ampliamente utilizado en aplicaciones web y resulta sencillo para equipos pequeños.

2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Permiten escalar partes específicas del sistema sin afectar a toda la aplicación. También facilitan el mantenimiento, las actualizaciones y el trabajo simultáneo de varios equipos de desarrollo.

3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

Separa claramente los datos, la interfaz y la lógica de control. Esto facilita el desarrollo, mantenimiento y crecimiento de las aplicaciones web.

4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

Podría aumentar los costos de desarrollo, requerir más tiempo de implementación y dificultar el mantenimiento. Además, se utilizarían recursos innecesarios para resolver problemas que el proyecto realmente no tiene.

5. ¿Crees que existe un patrón mejor que todos los demás?

No existe un patrón que sea mejor en todos los casos. La elección depende de las necesidades del proyecto, el número de usuarios, el presupuesto, el tamaño del equipo y los objetivos del sistema.

4.- CASO PRÁCTICO

Sistema seleccionado: Plataforma Educativa

Patrón seleccionado: Arquitectura en Capas combinada con MVC

Justificación

Número de usuarios:

Una plataforma educativa puede tener cientos o miles de estudiantes y docentes utilizando el sistema al mismo tiempo.

Escalabilidad:

La arquitectura en capas permite agregar nuevas funciones sin modificar completamente el sistema.

Complejidad:

La división en capas facilita la organización de módulos como cursos, usuarios, calificaciones y materiales educativos.

Mantenimiento:

Los errores y actualizaciones pueden realizarse en una capa específica sin afectar todo el sistema.

Costos:

Su implementación es menos costosa que una arquitectura basada en microservicios.

Tamaño del equipo de desarrollo:

Puede ser desarrollada eficientemente por equipos pequeños o medianos gracias a su estructura organizada.

Conclusión

Durante esta actividad aprendí que los patrones de arquitectura ayudan a organizar mejor un sistema y facilitan su desarrollo y mantenimiento. El patrón que más me llamó la atención fue MVC, ya que divide las funciones de la aplicación de manera clara y ordenada.

También comprendí que la arquitectura influye en aspectos importantes como el rendimiento, la escalabilidad y la calidad del software. Por ello, elegir una arquitectura adecuada desde el inicio del proyecto es fundamental para evitar problemas futuros y facilitar el crecimiento del sistema.